

Projects for the course *Algorithms for VLSI*

Jordi Cortadella

September 5, 2025

Abstract

Here is a short description of potential projects that could be selected for the course. The projects pose different problems in the area of Electronic Design Automation (EDA). The recommendation is to address the project from the early days of the course since they require a non-negligible effort. The students are also encouraged to propose other projects that could have some special interest for them, not necessarily in the area of EDA, but somehow related to the challenges faced in EDA problems. It is essential that the projects contain a significant algorithmic component.

1 Graph drawing

The project consists in implementing an algorithm for graph drawing (see https://en.wikipedia.org/wiki/Graph_drawing). The algorithm will have to read some simple specification of a graph (vertices and edges) and depict a good-quality pictorial representation of the graph after applying an algorithm that tries to create an understandable visualization of the graph.

Few strategies may be considered for graph drawing:

- Spectral methods using eigenvectors [Kor05]
- Multilevel force-directed methods [Hu06]
- Kamada-Kawai algorithm [KK89]

The benchmarks for graph drawing can be obtained from this [library](#).

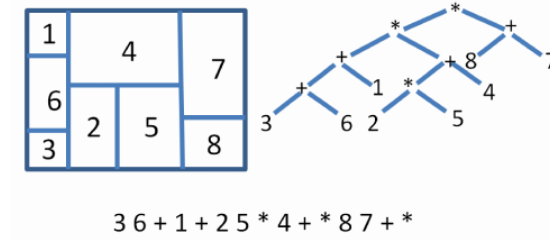


2 Floorplanning with slicing structures

The project consists in placing a set of blocks in a 2D surface. The quality of the layout will be determined by the area of the bounding box and the total wire length. See [https://en.wikipedia.org/wiki/Floorplan_\(microelectronics\)](https://en.wikipedia.org/wiki/Floorplan_(microelectronics)).

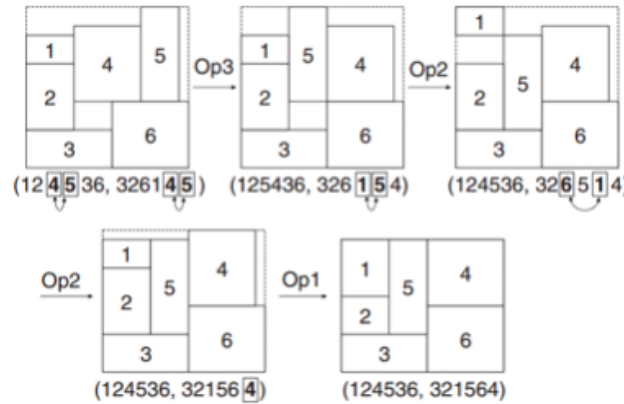
The final layout will have to be a sliceable floorplan (representable as a binary tree). The floorplanning algorithm will allow the blocks to be rotated according to some shape functions defined a priori.

A possible strategy for the implementation of the algorithm is described in [WL86].



3 Floorplanning with sequence pairs

Similarly to project 2, but changing the representation of the floorplan by a sequence pair, as described in [MFNK96] and shown in the figure.

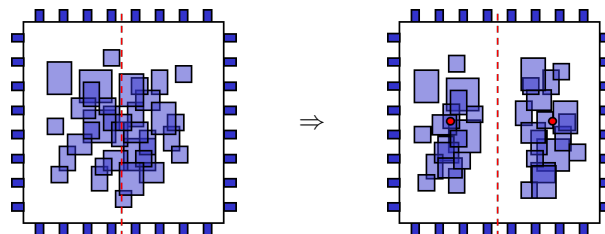


4 Global placement

Global placement is the step that precedes cell placement in VLSI design. The cells of the netlist must be uniformly distributed across the die in a way that the wirelength is minimized.

There are several strategies to do global placement. In particular the project should explore one of these two strategies:

- Min-cut placement, based on a recursive graph bipartitioning [Bre77].
- Analytical placement, based on a mathematical formulation of the problem [KGJA91].

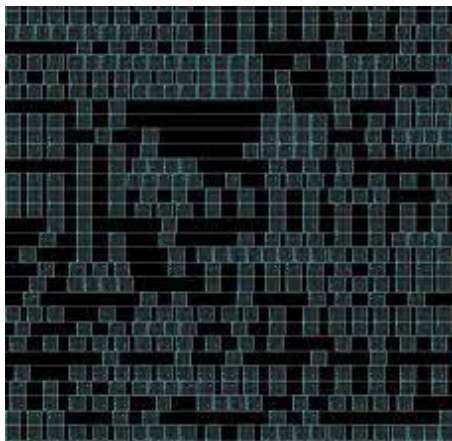


5 Cell placement

This project consists in placing the cells of a circuit in a 2D surface trying to minimize the total wire length (see [https://en.wikipedia.org/wiki/Placement_\(EDA\)](https://en.wikipedia.org/wiki/Placement_(EDA))).

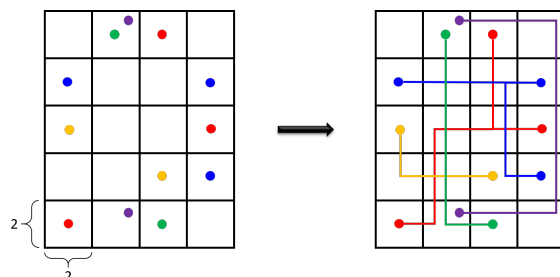
Conceptually, this project has some similarity to the project on graph drawing. The algorithm will have to read a netlist (a graph of logic cells) and position the nodes in a grid (an $n \times m$ matrix) in such a way that the total wire length is minimized. As a simplification, all cells will be assumed to occupy one slot in the matrix.

The project will have to use some meta-heuristics to solve the problem (e.g., Simulated Annealing [KGV83] or Extremal Optimization [BP03]).



6 Global routing

In this project, a global routing problem must be solved. The input of the problem is a grid and a set of points to be connected, as shown in the left figure. Each point has a color and the points with the same color must be connected.



To connect the points, wires across the cells of the grid must be created. There is one constraint: the boundaries of the cells have a certain capacity (2 in the example). This means that the number of wires that can cross every cell boundary must not exceed this capacity. The figure at the right shows a legal solution to the problem.

One of the goals of the project is to minimize the wire length. The solution in the example is not minimal. For solving the problem, an algorithm or a mathematical model must be proposed.

7 Detailed routing

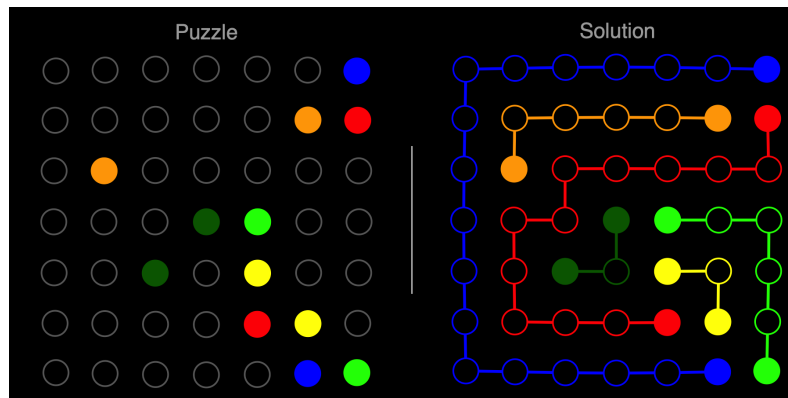
The project consists in designing an algorithm (or a mathematical model for a SAT or ILP solver), that generates routes in a 2D grid to connect pairs of dots. The input of the problem will be the

dimensions of a grid (e.g., 4×5), and a set of pairs of nodes, e.g., $\{(1, 1), (3, 2)\}$, $\{(2, 4), (3, 1)\}$, $\dots$. The algorithm will have to find routes along the edges of the grid that connect the pairs of nodes without creating any short circuit among them.

The project is a simplified version of a similar problem solved in this paper [CPGM14]:

https://www.cs.upc.edu/~jordicf/gavina/BIB/files/tcad2014_Routing.pdf

For more intuition, the problems to be solved are similar to the Flow Free game.



8 Two-level minimization

The project consists in implementing an exact algorithm for two-level minimization, i.e., finding the smallest sum-of-product form of a Boolean expression. The algorithm should be based on Quine-McCluskey method [McC56].

Given an expression representing a Boolean function, the algorithm will have to generate all prime implicants and select the smallest subset that covers all minterms of the function.

For example, given the following truth table

$abcd$	f
0000	0
0001	0
0010	-
0011	1
0100	0
0101	0
0110	0
0111	1
1000	0
1001	0
1010	0
1011	0
1100	1
1101	-
1110	1
1111	1

where '-' represents a *don't care* value, the minimizer should give the representation:

$$f = ab + \bar{a}cd$$

9 Dimensionality reduction for Boolean functions

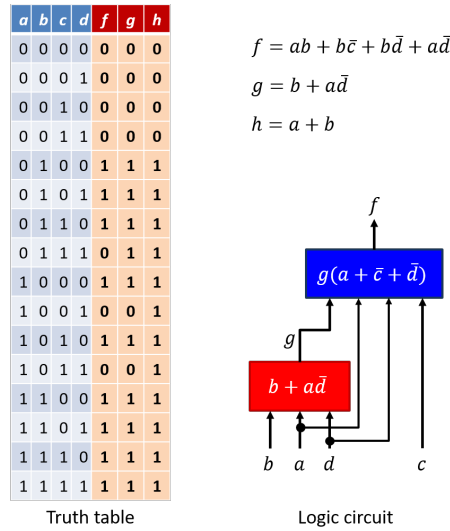
Logic Synthesis is the process of generating the implementation of a Boolean function using logic gates. A typical goal is to find a minimum-size implementation of the behavior of the Boolean function. When complex functions are to be implemented, the initial specification needs to be

decomposed into smaller functions that can be mapped into the basic primitives available in the technology, e.g., logic gates or lookup tables with few inputs.

Decomposition is a process that has to explore a large design space. A Boolean function can be decomposed in different ways and the selection of a good decomposition is essential to reduce the complexity of the final implementation. Therefore, having good estimators during decomposition contributes to take good decisions.

This project is about finding approximate representations of Boolean functions obtained by reducing the number of variables.

An example is shown in the figure. The left part shows the truth table of three 4-variable Boolean functions: f , g and h . Let us focus on the implementation of function f . A possible implementation in disjunctive normal form (DNF) is shown on the same figure (the Boolean equation has 8 literals).



Let us now consider approximations of f using a fewer number of variables. Function g only uses three variables and is the best approximation of f with this number of variables (notice that the only discrepancy is in the point $abcd=0111$). The question is: why is a,b,d the best subset of a,b,c,d to represent the function? How about using a,b,c or b,c,d instead?

As we drop more variables, the approximation is less accurate. For example, h is another approximation using only 2 variables, a and b . However, f and h differ in 3 points now: 0111, 1001 and 1011.

The conjecture is that good approximations of Boolean functions contribute to find good decompositions, as shown in the circuit of the figure. By using function g (red box), a simple implementation of f can be found (blue box).

The challenge The main question we want to answer is the following: given an n -variable Boolean function, what is the subset of k variables that can be used to find the best approximation of the function? For example, imagine a 20-variable function and an approximation using only 6 variables.

10 Algebraic factorization

The project consists in implementing an algorithm to find a good factorization of a Boolean expression using algebraic methods. For example, given the expression:

$$F = adf + aef + bdf + bef + cdf + cef + bfg + h$$

the factorization algorithm could find a multi-level expression such as this one:

$$F = (a + b + c)(d + e)f + bfg + h.$$

One of the popular algorithms for factorization is based on kernel extraction [Bra87] and reduces the problem to finding rectangles in a Boolean matrix. Another possible strategy is based on Graph Partitioning [GM99].

11 Deep learning for predicting the size of circuits implementing Boolean functions

This is a novel project that aims at estimating the complexity of implementing a Boolean function by means of supervised learning using the truth tables of a function.

Given a number of variables (e.g., $n=10$), the project consists of generating random functions and synthesizing the functions using logic synthesis tools. The results will be used to train a neural network with the goal of predicts the number of 2-input gates required to implement the function.

Comment: An extended version of this project could be the basis of a Master thesis.

Resources

Here are some resources that can be used for the projects.

- SpyDrNet: A flexible framework for analyzing and transforming netlists [Spy].
- PyEDA: A Python library for electronic design automation [PyE].
- KaHyPar: High-Quality Multilevel Hypergraph Partitioning [KaH].
- Yosys: A framework for Verilog RTL synthesis [Yos].
- NetworkX: Network Analysis in Python [Net].

References

- [BP03] Stefan Boettcher and Allon G. Percus. *Extremal Optimization: An Evolutionary Local-Search Algorithm*, pages 61–77. Springer US, Boston, MA, 2003.
- [Bra87] R. K. Brayton. Factoring logic functions. *IBM Journal of Research and Development*, 31(2):187–198, 1987.
- [Bre77] Melvin A. Breuer. A class of min-cut placement algorithms. In *Proceedings of the 14th Design Automation Conference, DAC '77*, page 284–290. IEEE Press, 1977.
- [CPGM14] Jordi Cortadella, Jordi Petit, Sergio Gómez, and Francesc Moll. A Boolean Rule-Based Approach for Manufacturability-Aware Cell Routing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(3):409–422, March 2014.
- [GM99] M. C. Golumbic and A. Mintz. Factoring logic functions using graph partitioning. In *IEEE/ACM International Conference on Computer-Aided Design*, pages 195–198, 1999.
- [Hu06] Yifan Hu. Efficient, high-quality force-directed graph drawing. *The Mathematica journal*, 10:37–71, 2006.
- [KaH] KaHyPar: High-Quality Multilevel Hypergraph Partitioning. <https://kahypar.org>.
- [KGJA91] J. M. Kleinhans, G. Sigl, F.M. Johannes, and K.J. Antreich. Gordian: VLSI Placement by Quadratic Programming and slicing Optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pages 356–365, March 1991.
- [KGV83] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [KK89] T. Kamada and S. Kawai. An algorithm for drawing general undirected graphs. *Information Processing Letters*, (31):7–15, 1989.
- [Kor05] Y. Koren. Drawing graphs by eigenvectors: theory and practice. *Computers & Mathematics With Applications*, 49:1867–1888, 2005.
- [McC56] E. J. McCluskey. Minimization of boolean functions. *The Bell System Technical Journal*, 35(6):1417–1444, 1956.
- [MFNK96] H. Murata, K. Fujiyoshi, S. Nakatake, and Y. Kajitani. Vlsi module placement based on rectangle-packing by the sequence-pair. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 15(12):1518–1524, 1996.

- [Net] NetworkX: Network Analysis in Python. <https://networkx.org>.
- [PyE] PyEDA: a Python library for electronic design automation. <https://pyeda.readthedocs.io>.
- [Spy] SpyDrNet. <https://byuccl.github.io/spydrnet/docs/stable/index.html>.
- [WL86] D. F. Wong and C. L. Liu. A new algorithm for floorplan design. In *23rd ACM/IEEE Design Automation Conference*, pages 101–107, 1986.
- [Yos] Yosys: a framework for Verilog RTL synthesis. <https://yosyshq.net/yosys/>.